

Saé S1.01 : implémentation d'un besoin client

Les tableaux à une dimension en C (tableaux 1D)

1. Définitions

Un tableau à une dimension (tableau 1D), aussi appelé vecteur, est une structure de données bornée qui permet de stocker plusieurs valeurs de même type. Un tableau peut être vu comme une succession de "cases" contenant les valeurs. Par abus de langage, les valeurs sont aussi appelées éléments du tableau.

Les cases du tableau sont numérotées dans l'ordre. Ces numéros sont appelés "indices".

Attention ! En langage C cette numérotation commence toujours à 0. Un tableau de N cases aura des cases indicées de 0 à N-1.

Exemple

Un tableau de 5 réels peut se représenter comme ceci :

indices du tableau :	0	1	2	3	4
valeurs (ou éléments) du tableau :	13.50	12.50	17.00	9.75	13.25

1.1. Déclaration d'un tableau 1D

Lors de la déclaration d'une variable tableau, il faut indiquer :

- sa longueur maximale (*ie* son nombre total de "cases") entre crochets,
- le type de ses éléments.

Exemple

```
int monTableau[10];
```

Ici, la variable *monTableau* contient 10 cases pouvant contenir des valeurs entières. Ces cases sont indicées de 0 à 9.

Mais bien sûr, il est préférable de déclarer la taille maximale du tableau en constante.

```
#define TAILLE 10  
puis  
int monTableau[TAILLE];
```

1.2. Accès à une valeur/modification d'une valeur

On l'a vu, chaque valeur dans le tableau est associée à un indice. Pour manipuler une valeur d'un tableau, il faut indiquer son indice entre crochets derrière le nom de la variable tableau.

Exemple :

<i>monTableau</i>	13.50	12.50	17.00	9.75	13.25
-------------------	-------	-------	-------	------	-------

Avec ce tableau, modifier la 3^{ème} valeur se programme comme ceci :

```
monTableau[2] = 18.00;
```

Et pour afficher à l'écran la 4^{ème} valeur :

```
printf("%f", monTableau[3]) ;
```

1.3. Initialisation d'un tableau

La seule possibilité d'initialiser les valeurs d'un tableau statiquement (c'est-à-dire avant l'exécution), c'est au moment de sa déclaration. Il faut alors fournir, entre accolades, la liste exhaustive des valeurs.

Exemple :

```
#define TAILLE 5
...
int leTableau[TAILLE] = {1, 3, 5, 7, 9};
```

On peut aussi écrire (mais c'est déconseillé) :

```
int leTableau[] = {1, 3, 5, 7, 9};
```

la taille du tableau étant automatiquement calculée en fonction de la liste des valeurs.

Exercice 1

Dans un programme, déclarez un tableau de 12 entiers, initialisé avec les 12 premiers nombres impairs. Dans le main, ajoutez 4 à la 5^{ème} valeur et retranchez 4 de la 9^{ème} valeur. Puis affichez ces deux valeurs à l'écran.

Exercice 2

Dans un programme, déclarez un tableau de 10 caractères, initialisé avec les dix premières lettres de l'alphabet. Dans le main, affichez à l'écran les valeurs de la première moitié de ce tableau : a b c d e

1.4. Déclaration d'un type tableau

Lorsque dans un programme on souhaite utiliser plusieurs tableaux de même nature (même taille et valeurs de même type), il peut être intéressant de créer un nouveau type. En C, cela est possible avec le mot clé `typedef`.

Exemple

Pour créer un nouveau type représentant tous les tableaux de 5 entiers, on écrit :

```
typedef int t_tableau[TAILLE];
```

Il devient alors possible de déclarer des variables de type `t_tableau` :

```
t_tableau monTablo;
```

Exercice 3

Reprenez les exercices 1 et 2 en déclarant un type tableau.

1.5. Passage de tableaux en paramètre

ATTENTION : un tableau est toujours un paramètre d'entrée/sortie. Inutile donc d'utiliser `*` dans la définition des procédures et fonctions et inutile d'utiliser `&` lors de l'appel.

2. Algorithmes de base

2.1. Parcours complet des valeurs d'un tableau

Il est souvent utile de parcourir l'ensemble des valeurs d'un tableau, du premier au dernier élément. Pour cela, on utilise cette boucle FOR :

```
for (i = 0 ; i < MAX ; i++){  
    //utilisation ou modification de tab[i]  
}
```

Attention à ne pas inclure
MAX dans la boucle.
tab[MAX] n'existe pas !

Exercice 4

Dans un nouveau programme, déclarez un nouveau type pour des tableaux de 10 entiers.

Écrivez une procédure `initialise(...)` qui initialise les 10 valeurs du tableau passé en paramètre avec des entiers lus au clavier.

Écrivez une procédure `affiche(...)` qui affiche les 10 valeurs du tableau passé en paramètre

Testez ces deux procédures dans un `main`.

2.2. Recherche séquentielle dans un tableau

L'algorithme qui suit permet de rechercher la présence d'une valeur v dans un tableau `tab` :

```
trouve := FAUX;
i := 0;
tant que (non trouve ET i < MAX) faire
    si (tab[i] == valeur) alors
        trouve := VRAI;
    sinon
        i := i + 1 ;
fin_si
fin_tant_que
```

Exercice 5

Dans le programme de l'exercice précédent, ajoutez une fonction booléenne `existe(...)` qui prend un tableau de 10 entiers et un entier en paramètres et qui retourne `true` si l'entier est présent dans le tableau et `false` sinon. Testez-la dans le `main`.

Exercice 6

Dans le même programme, transformez votre fonction `existe(...)` pour qu'elle retourne l'indice de la première occurrence trouvée, ou bien qui retourne `-1` si la valeur est absente du tableau. Testez cette nouvelle version dans le `main`.

3. Tableaux 1D partiellement remplis

Le principal problème des tableaux c'est qu'ils sont bornés, c'est à dire que leur taille est figée dès la déclaration. Mais bien souvent on a besoin de stocker des valeurs de même type sans connaître forcément le nombre de ces valeurs à l'avance. Alors on dimensionne le tableau en "prenant de la marge", en indiquant une taille maximale mais en n'utilisant qu'une partie seulement du tableau.

Exemple1

Un tableau pour stocker les épreuves d'une matière :

DS1	TP1	TP2		
-----	-----	-----	--	--

Actuellement il y a 3 épreuves, mais il sera possible d'en stocker jusqu'à 5 (par contre, il sera impossible d'en ajouter une sixième).

Important : dans ce cas il faut alors distinguer la taille maximale du tableau (nombre total de "cases") et la taille réellement utilisée (nombre de "cases" réellement occupées).

Exemple 2

Pour stocker le nom d'un étudiant : tableau de 20 caractères...

L E _ C L O A R E C \0

...mais bien sûr, suivant le nom de l'étudiant, seule une partie du tableau est utilisée.

(vous noterez qu'en C, une chaîne est en fait un tableau de caractères dont le dernier élément est un caractère spécial '\0', qui est appelé marqueur de fin de chaîne)

4. Exercices d'application

4.1. Exercice : "vecteurs"

On considère les déclarations suivantes :

```
#define N 5
typedef int vecteur[N]; //ici, un vecteur est un tableau de 5 entiers

void remplirVecteur (vecteur v);1
// initialise v avec les valeurs fournies au clavier .

void afficherVecteur (vecteur v);
//affiche à l'écran les N coefficients du vecteur v.

void sommeVecteur(vecteur v1, vecteur v2, vecteur vSomme);
// met dans vs la somme des vecteurs v1 et v2.
```

Question 1

Écrivez ces trois procédures.

Question 2

Écrivez le *main* qui remplit deux vecteurs, en fait la somme et affiche le vecteur résultant à l'écran.

1 Le paramètre formel de type tableau peut être déclaré comme `vecteur` (dans cet exemple) ou bien comme `int[]` ou bien comme `int*`.

4.2. Exercice 2 : "températures"

Dans un hôpital, on relève la température d'un patient chaque jour et on stocke ces valeurs sur les n derniers jours, n étant saisi au début du programme. Écrivez un programme qui permet la saisie des températures d'un patient puis qui affiche la température moyenne de ce patient et qui indique si le patient va mieux, moins bien ou si son état est stable selon que sa température a baissé, a augmenté, ou bien est la même sur les deux derniers jours.

Exemple d'exécution du programme (en gras les saisies de l'utilisateur)

```
Nombre de jours ? 4
température jour 1 ? 39.4
température jour 2 ? 39.5
température jour 3 ? 38.4
température jour 4 ? 38.2

Température moyenne du patient sur les 4 jours : 38.9
Le patient va mieux.
```

Question 1

Dans un nouveau programme C, définissez les constantes, types et variables nécessaires pour le stockage des températures d'un patient.

Question 2

Écrivez une procédure pour l'initialisation du tableau de températures par les lectures au clavier. Le tableau ainsi que le nombre de valeurs à saisir sont passées en paramètres.

Question 3

Écrivez une fonction qui calcule et retourne la moyenne des valeurs d'un tableau de températures. Le tableau ainsi que le nombre de valeurs à saisir sont passées en paramètres.

Question 4

Écrivez un main qui lit au clavier un nombre de valeurs puis qui affiche la moyenne de ces valeurs et qui indique l'état du patient ("va mieux", "va moins bien" ou "est stable").

Note : la valeur de la moyenne sera affichée avec une seule décimale.

4.3. Exercice 3 : "notes de DS"

On veut connaître la répartition des notes d'un DS, c'est-à-dire connaître le nombre de 0, le nombre de 1, ..., le nombre de 20, ayant été attribués à ce DS.

En **utilisant les procédures demandées ci-dessous**, écrivez un programme lisant au clavier une suite de notes (une note est un entier compris entre 0 et 20) terminée par -1, et affichant le nombre d'occurrences de chacune des notes possibles ().

Le programme devra utiliser :

- une procédure pour initialiser le tableau *nb*,
- une procédure pour mettre à jour le tableau *nb*,
- une procédure pour afficher les résultats. Pour améliorer la lisibilité du résultat, on ne fera un affichage que pour les notes ayant au moins une occurrence.

Notes

- Les données ne sont pas stockées dans un tableau, mais traitées au fur et à mesure de leur lecture.
- On utilisera un tableau *nb* tel que *nb[note]* sera le nombre d'occurrences de la note *note*.

4.4. Exercice 4 : "à la banque"

Au guichet d'une banque, les clients retirent de l'argent liquide. La somme en euros est remise au client en pièces et billets, en minimisant leur nombre.

Écrivez un programme, **utilisant les procédures demandées ci-dessous**, qui pour chaque client demande son nom et la somme que celui-ci retire. La fin de la saisie est indiquée par la présence d'une étoile à la place du nom du client.

Le programme affiche, pour chaque client, au fur et à mesure de la saisie, le nombre de pièces et billets de chaque sorte à lui remettre, puis à la fin, affiche pour l'ensemble des clients le total de chaque sorte de pièce et billet.

Note

Les données ne sont pas stockées dans un tableau, mais traitées **au fur et à mesure** de leur lecture. **Les tableaux nécessaires (et suffisants) sont donc :**

- une constante tableau *valeurs* contenant les valeurs des pièces et billets.
- un tableau *total* tel que *total[i]* sera le nombre total de pièces ou billets de valeur *valeurs[i]* pour l'ensemble des clients.

Le programme devra utiliser des procédures :

- une procédure pour initialiser le tableau *total*,
- une procédure pour décomposer la somme (avec affichage de la décomposition) et donc mettre à jour le tableau *total*
- une procédure pour afficher le récapitulatif pour l'ensemble des clients

On considèrera uniquement les pièces ou billets suivants :

1, 2, 5, 10, 20, 50, 100, 200, 500 .

Exemple d'exécution souhaitée (en gras les valeurs saisies par l'utilisateur) :

Nom ? : **Toto**
Somme ? : **840**

Toto :
1 x 500
1 x 200
1 x 100
2 x 20

Nom ? : **Lili**
Somme ? : **1750**

Lili:
3 x 500
1 x 200
1 x 50

Nom ? : **Momo**
Somme ? : **18**

Momo :
1 x 10
1 x 5
1 x 2
1 x 1

Nom ? : *****

Total pour l'ensemble des clients :
4 x 500
2 x 200
1 x 100
1 x 50
2 x 20
1 x 10
1 x 5
1 x 2
1 x 1